

Boosting Energy Efficiency and Quality of Service through Orchestration Tools

Alessandro Carrega
Consorzio Nazionale
Interuniversitario per le
Telecomunicazioni (CNIT) –
S3ITI National Lab

Giancarlo Portomauro
University of Genoa

Matteo Repetto
CNIT – S3ITI National Lab

Giorgio Robino
CNIT – S3ITI National Lab

In this paper, we describe a novel paradigm to pre-provision virtual machines (VMs) in an energy-efficient manner for cloud elastic applications. Our approach is based on the definition of dynamic context for VMs, which can be easily updated by software orchestration tools. We show the feasibility of our approach and improvement over existing state of the art with an experimental setup.

High availability and service continuity—especially for mission-critical applications—often require deployment of more software instances than strictly needed, which could be used in the case of sudden peaks of requests, failure of the primary components, or movement of users to a different location. Sometimes, as happens for billing and other periodic administrative tasks, resources are permanently provisioned to run applications for a few days per month. In the aforementioned cases, users are therefore forced to pay for unused cloud resources, while energy is wasted in the infrastructure to keep up servers that host idle applications.

In this paper, we discuss a new cloud management paradigm for the Infrastructure-as-a-Service (IaaS) model, which provides better balance between energy efficiency and Quality of Service (QoS). Our approach is based on the introduction of more context information by users, who explicitly declare whether their virtual machines (VMs) are used or idle. The knowledge of context information allows cloud providers to improve their management practice and save more energy in the infrastructure. In turn, the savings would result in new pricing schemes that reward users for declaring context information. Software orchestration enables the automation of most operations that are needed to manage the dynamic context.

The realization of the above paradigm builds on two main elements, which are still missing in the state of the art. First, extensions to current cloud management software are required to manage context information, QoS, and energy efficiency. Second, consolidation and management algorithms should be aware of the presence of idle resources and place them so as to selectively apply aggressive power-saving mechanisms. We have already designed a workload consolidation algorithm that exploits context information, which has been evaluated through simulations.¹ More recently, we have also designed and implemented a set of OpenStack extensions to manage QoS and energy efficiency, although evaluation was limited to reporting power profiles of real equipment.² This paper extends our previous work by (i) investigating the trade-off between QoS and power consumption on an experimental testbed and (ii) implementing and evaluating our consolidation strategy in the same testbed, so as to draw out more accurate and realistic estimation of energy savings with context information.

RELATED WORK

Power efficiency in data centers is largely dependent upon a linear relationship between power and performances,³ which is usually pursued by leveraging existing power-saving mechanisms: performance scaling (such as dynamic voltage/frequency regulation for CPUs) and, most of all, removal of idle periods. Under this premise, running VMs on the smallest number of servers is often the best solution to save energy.⁴ This process is conventionally indicated as workload “consolidation.”

The consolidation problem has often been modeled as a modified bin packing problem;⁴ however, VM placement is not exactly equivalent to a 3D (CPU, RAM, I/O) bin packing problem, and some anomalies could arise from this approach and similar ones in the literature.⁵ Since the ultimate objective for consolidation is energy minimization for the whole infrastructure, the consolidation strategy should account for both computing and networking devices.⁶⁻⁹

Similar to our purpose, Shirayanagi *et al.* considered the presence of backup replicas and links, and investigated through simulations how power-consumption changes when varying their number.⁶ However, the focus was on the power consumption of the network alone. Li *et al.* investigated the impact of overprovisioning (such as for dealing with sudden peaks of workload) on power consumption.¹⁰ Their strategy for workload consolidation considered only the servers and did not take into account any network metric.

Despite the large number of algorithms proposed in the literature, the feasibility, effectiveness, and efficiency of workload consolidation in real testbeds have not been fully investigated yet. Heller *et al.* analyzed how redundancy and guard margins influence the power consumption; their work places VMs with the objective to minimize the power consumption of the data center network, but does not consider computing servers.¹¹ More recently, Beloglazov *et al.* proposed OpenStack Neat for balancing QoS and energy consumption.¹² Their framework monitors CPU utilization by VMs, estimates overloading/under-loading conditions on each server, and uses OpenStack APIs for life-migration of VMs between servers. Their consolidation strategy is power-unaware, since only the utilization metric is used.

Our approach improves existing state of the art by including a power monitoring framework, software-defined networking (SDN), and specific context for applications. SDN allows fine-grained control over network traffic streams, including bandwidth reservation. Power monitoring improves consolidation decisions with respect to simple linear or step-wise models, which cannot take into account the heterogeneity of the underlying hardware and the complexity brought by the presence of multiple CPUs, multiple cores, and power-management features (voltage/frequency scaling, low-power idle, and so on).¹³

INTRODUCING MORE CONTEXT FOR ENERGY EFFICIENCY AND QOS

The high degree of automation present in much cloud management software allows users to dynamically grow and shrink their resources (such as CPU, RAM, disk, and network) within pre-

defined quotas, so as to tackle occasional conditions as failure of some components or workload variations (the pay-as-you-go model). Software orchestration is emerging as a management paradigm to automate and accelerate most operations that must be performed for reliable, continuous, resilient, and efficient operation, including continuous monitoring of operational parameters and events, resource provision, software deployment, and configuration. However, the time to detect (or predict) changes, to provision resources, to deploy the software, and to re-configure the application is not negligible (on the order of several minutes); this is enough to jeopardize QoS and service level agreements (SLAs), especially for the most demanding and critical applications.

To avoid violations of QoS and SLAs, especially for critical business applications that require high availability and service continuity, the only viable solution today is pre-provision of additional (idle) resources, which remain always ready to serve sudden peaks in the workload or to replace failed, malfunctioning, or compromised instances. That means cloud users are somehow forced to pay for something that might only be used occasionally. In addition, energy efficiency of the underlying infrastructure and the ideal linear relationship between power and performances are partially undermined by the presence of idle instances.

QoS and Context Information

We argue that most of the (energy) inefficiency in current data center management comes from the lack of knowledge about applications, their requirements, and their status. Cloud management software does not know what each software instance is meant for; therefore, every instance must be treated the same way and with the same QoS guarantees. In general terms, the context for each software unit (VM) should include the following kinds of information:

- *usage patterns*, reporting average and/or maximum utilization values for CPU, RAM, disk, and network bandwidth;
- *service level*, related to the criticalness of the service and the willingness of users to accept some degree of violations of QoS; and
- *availability*, indicating whether the component will be used in the next timeframe.

Usage patterns consist in numerical values that can be easily added to any cloud management software. For service level and availability, a specific model should be defined. In our work, we use a very simple yet powerful model, based on a few colored labels: red, yellow, and green.

Red labels are assigned to VMs that host the critical software, with strict QoS constraints on processing, memory, availability, and bandwidth requirements. As compensation for higher fares, users expect no over-commitment and no live migration to occur in this case. For instance, an entire physical core may be dedicated to each virtual CPU, while network bandwidth is reserved according to the maximum declared throughput. Yellow labels are assigned to standard VMs, for which over-commitment is acceptable. For example, the average declared value for CPU utilization may be used to compute the over-commitment ratio, while network reservation may use average or equivalent bandwidth values. Finally, green labels are assigned to idle and unused VMs. They can be subject to extreme over-commitment and aggressive power-saving mechanisms, provided they can return available with very low delay (below a few seconds). This simple model could be easily extended to more classes and a richer semantics for QoS.

All the described parameters could be set manually, but this would require some time to change the indicator of availability (to switch between the green and yellow label). Software orchestration greatly facilitates the task by automating operations through the definition of policies. For example, a scaling policy may activate additional instances (set label to yellow) when the response latency exceeds a given threshold, and deactivate some instances (set label to green) when the same latency falls below another threshold.

Energy Efficiency and Workload Consolidation

The introduction of more context for cloud applications is just the enabler for improving the effectiveness of workload consolidation. The improvement in energy efficiency comes from this

consideration: cluster unused VMs together and put the physical server that hosts them in extreme low-power state, namely ACPI S3, known as suspend-to-ram (we use the colloquial term “sleep”). This power state is very efficient, because it consumes just a few Watts even on most powerful servers, while being able to restore to full operation in a few hundreds of milliseconds.

Suitable consolidation strategies should be devised to find the proper placement of red, yellow, and green VMs in the data center, so as to avoid performance degradation and the need for too many migrations. As a matter of fact, green VMs are unused and do not consume resources. However, if they are placed with other red/yellow instances, their activation might affect the QoS of such co-located instances.

The usage of SDN for packet forwarding and bandwidth reservation improves the utilization of redundant communication infrastructures and brings further opportunities for energy savings. In fact, network traffic can be consolidated on the smallest number of links and switching equipment, while all other devices are put to sleep.

A FRAMEWORK FOR ENERGY EFFICIENCY

To demonstrate the main concept behind our work, we have designed and implemented:

- an energy-efficient infrastructure that gives full control over the power state of computing and networking devices and
- a consolidation algorithm that places VMs and manages the power state of the underlying infrastructure.

Energy-Efficient Infrastructure

We designed an energy-efficient infrastructure by developing a set of extensions to plain OpenStack, which include:

- a software-defined network with power-saving mechanisms;
- a power controller, which collects measurements about power consumption and CPU utilization and manages power states of servers and network devices; and
- management interfaces.

Figure 1 shows the conceptual architecture of our energy-efficient infrastructure. The North interface is conceived for the users, while the East interface is used by infrastructure providers for control and management.

The North interface is the plain OpenStack user API, which we use to set context information. We set CPU usage, bandwidth requirements, and label color as `<key,value>` pairs associated to each VM. In addition, to avoid unreachability errors when physical servers are put to sleep, we expect that users make use of the OpenStack PAUSED state for green VMs (this is a software abstraction of ACPI S3).

The East interface includes the cloud administration API, the power management interface, and the SDN northbound interface. The cloud API is the standard OpenStack interface, used in admin mode to retrieve information about the virtualization system (running VMs, state, tenant networks, VLAN ids, context metadata, and so on) and to trigger migration of VMs. The power management interface retrieves information about current power consumption and changes the power state of servers and network devices. Power saving is currently based on two ACPI states: active (ACPI S0) and sleep (ACPI S3). Finally, the SDN northbound allows programmatic configuration of the communication infrastructure by RESTCONF, which provides high-level network abstraction through the Yang model. Full implementation details about the design and implementation of the energy-efficient infrastructure are already given in our previous work.²

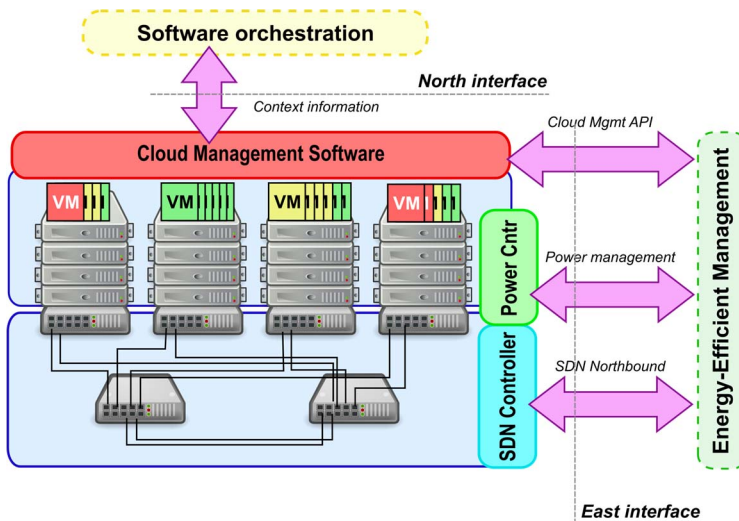


Figure 1. Conceptual architecture for energy-efficient infrastructure. Dotted boxes are external elements that interact through North and East interfaces.

Workload Consolidation

We modeled workload consolidation as an optimization problem. The problem is formulated as a Greedy Bin Packing with approximation (GBP) for placement of VMs and multi-commodity flow (MCF) for network paths. Despite the complexity and the dimension of the problem, we followed a different approach than other related works in the literature.^{5,7,9} Instead of solving two problems separately (GBP and MCF), we formulate a global optimization procedure, which solves the placement and routing problems simultaneously. The objective function to be minimized is the sum of the power consumption of all infrastructure devices, including computing servers and networking equipment; the decision variables are the placement of VMs on the physical servers and the network paths. The model also includes optimization constraints, which keep into account the capacity of servers and network links (also considering different over-commitment ratios for red, yellow, and green VMs) and logical relationships between network traffic and placement of VMs. We impose arbitrary limits on the number of migrations and power state transitions, to avoid oscillatory behaviors that waste resources and weaken the service level. Full description and mathematical formulation is available in our previous work.¹

Since the problem is NP-hard, we use the heuristics available in the OptaPlanner solving tool to find good solutions in a reasonable amount of time. The result of the consolidation typically clusters together red/yellow instances, whereas green instances are left aside and do not need to be consolidated (all these servers are put to sleep, so the distribution of green VMs among them does not affect the overall power budget).

Finally, we point out that a unique feature of our consolidation algorithm is the usage of real power consumption for all physical devices. This allows far better precision in the consolidation strategy than any other proposed power consumption model.^{2,14}

EXPERIMENTAL RESULTS

We investigate some practical aspects about the relationship between performance and power consumption that are often underestimated in the literature. The purpose of our experimental analysis is twofold:

- to investigate how more or less aggressive consolidation affects power consumption and the service level (consolidation vs. performance) and
- to compare energy savings when context information is known with the legacy behavior (consolidation and energy savings).

The layout of our testbed is depicted in Figure 2. It is a small installation, clearly not representative of a real data center, but it is enough for functional validation and to understand the QoS/energy-savings potential behind our approach.

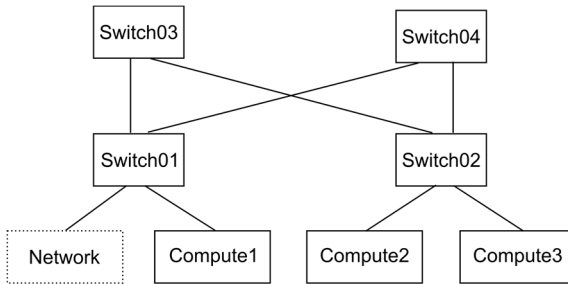


Figure 2. Testbed topology and functional elements.

The cloud software is the OpenStack Newton release, while for the SDN controller, we used the Carbon release of OpenDayLight. We used a mix of bash scripts and simple Java programs for implementing extended APIs and functionalities (the software to retrieve and manage power data, as well as to change the ACPI power state). Power consumption is measured by a Raritan PX3-5260R Power Data Unit, and measurements are accessible by SNMP.

Consolidation vs. Performance

We investigated how consolidation affects performance and power consumption. To this aim, we had six VMs, and we identified three placement scenarios. Each scenario corresponds to a more or less aggressive consolidation behavior and degree of over-commitment. It basically represents a sort of “snapshot” of a consolidation strategy, to focus on placement decision while neglecting all aspects concerning migrations and transitions. The placement of VMs for each scenario is described in Table 1.

Table 1. Allocation of VMs for each scenario under consideration.

Scenario	Compute1	Compute2	Compute3
1. Max saving	All VMs (VM1-6)	–	–
2. Balanced	3 VMs (VM1,3,5)	3 VMs (VM2,4,6)	–
3. Max performance	2 VMs (VM1,4)	2 VMs (VM2,5)	2 VMs (VM3,6)

We used the stress-ng tool to incrementally increase the CPU load for each VM. We compare power consumption and performances for the three scenarios in Figure 3. We see in the left side that a large power-consumption gap is present between Scenario 1 and Scenario 2 (around a 40-percent increase), while the difference is limited between Scenario 2 and Scenario 3. Indeed, by using Compute2 in addition to Compute1, we need to wake up two additional switches (Switch02 and Switch03), which are not necessary in Scenario 1. No additional network devices are necessary when switching from Scenario 2 to Scenario 3.

Even though Scenario 1 achieves greater energy savings, we must also take performance into account for fair comparison. To this aim, we consider the number of operations per seconds, as reported by stress-ng metrics for full-load experiments (all VMs running at 100-percent CPU). We note in the right side of Figure 3 that performances are almost constant while running one, two, or three VMs concurrently, but they significantly degrade for six VMs. This is not surprising, since each processor on the compute nodes has four cores with hyper-threading; although they appear as eight vCPUs to the OS, hyper-threading is not equivalent to having additional real cores.

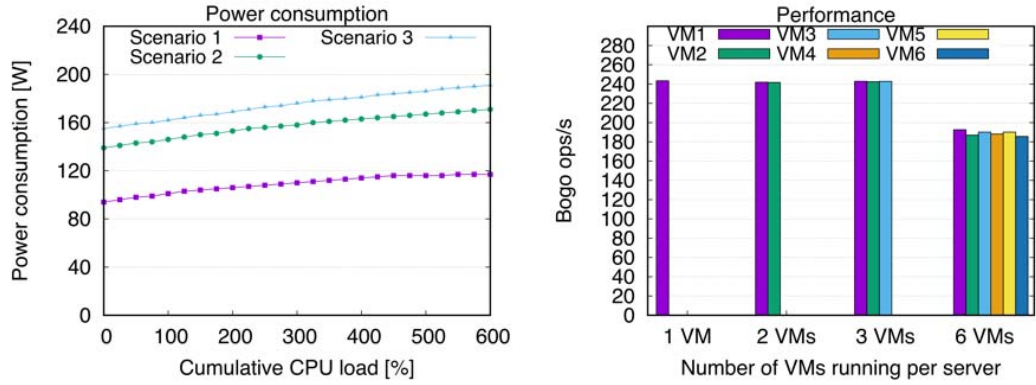


Figure 3. Power consumption and performances for different allocation scenarios.

For better evaluation, we compare in Figure 4 the variation of three relevant indexes in the considered scenarios, arbitrarily assuming Scenario 1 as base reference (100 percent): performance, power, and overall efficiency. We take into consideration three levels of CPU utilization for VMs: 100 percent (full load), 75 percent (high load), and 50 percent (medium load).

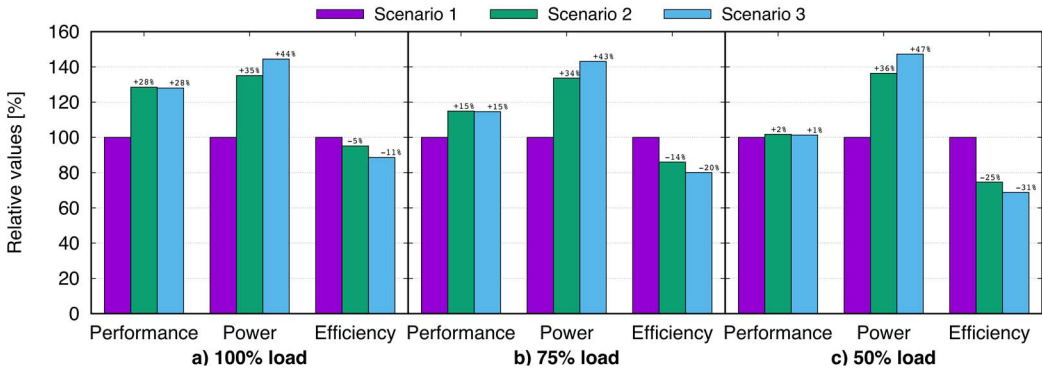


Figure 4. Comparison of performance (total operations per second), power consumption, and overall efficiency (operations over power consumption ratio) for the three consolidation scenarios, under different utilization factors.

Figure 4 shows that, for full-load utilization, Scenarios 2 and 3 have almost identical performance, but different power consumption. Scenario 1 is undoubtedly the most efficient solution, though this comes at the cost of worse performances. We can say that Scenario 2 is a better placement strategy than Scenario 3, since we have better efficiency and lower power consumption with the same performance level. Instead, comparison between Scenario 1 and Scenario 2 is not straightforward, because it depends on the overall optimization objectives (which may lean towards performance or energy savings). In general, Scenario 1 might be preferable, since efficiency is higher with respect to the other scenarios.

When VMs need less CPU time (lower utilization level), there is more room for over-commitment. Indeed, the performance increment for Scenarios 2 and 3 is limited to 15 percent with 75-percent CPU load and negligible with 50-percent CPU load. Instead, the power increment is comparable with the previous case, and this turns into greater loss of efficiency from Scenario 1 to Scenario 3.

We conclude that more aggressive consolidation (Scenario 1) is the optimal choice when VMs are not 100-percent loaded. Instead, when VMs are fully loaded, less aggressive consolidation brings larger performance gains, which motivate a slight loss of efficiency. In general, less aggressive consolidation is better suited for latency-sensitive and mission-critical applications, where performance should prevail over power efficiency.

Consolidation and Energy Efficiency

We now consider the dynamic behavior when consolidation is applied to an elastic application. To this purpose, we developed a very simple video transcoding service made of one control master (“dispatcher”) that delegates transcoding tasks to several slaves (“workers”). Since orchestration is not the primary objective of our investigation, we used a very dumb policy, which wakes a worker up when a new request arrives and pauses it when it returns the result.

The consolidation algorithm is run both periodically and when a new task arrives. To limit the computation burden, the algorithm is not executed when an existing request is completed. In this case, a sub-optimal condition will persist until the next periodic execution. The optimization plan elaborated by the consolidation algorithm consists of (i) the placement of the VMs on computing servers and (ii) the identification of forwarding paths and bandwidth reservation in the physical network.

We chose the request arrival rate so as to reproduce a larger and a smaller peak, as shown in Figure 5, and we repeated the same experiment until results were in the 95-percent confidence interval. All the results described in this section are the average of the repeated experiments.

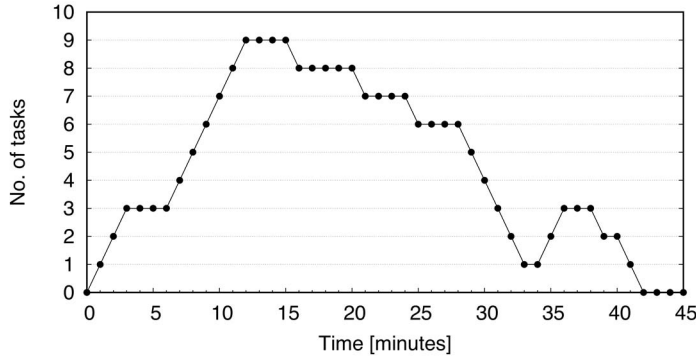


Figure 5. Workload profile (number of transcoding tasks during the experiment).

We compare the results in three scenarios: (i) without consolidation, (ii) with consolidation but not taking into consideration context information, and (iii) with consolidation using context information. For the second scenario, we used our consolidation algorithm without any constraint on label color; this allows for a fair assessment of the impact of using context information.

Figure 6 shows and compares power consumption (left side) and power savings (right side) from the experiments. The utilization shape (corresponding to the workload profile) is reflected in power consumption, which increases when more tasks are present.

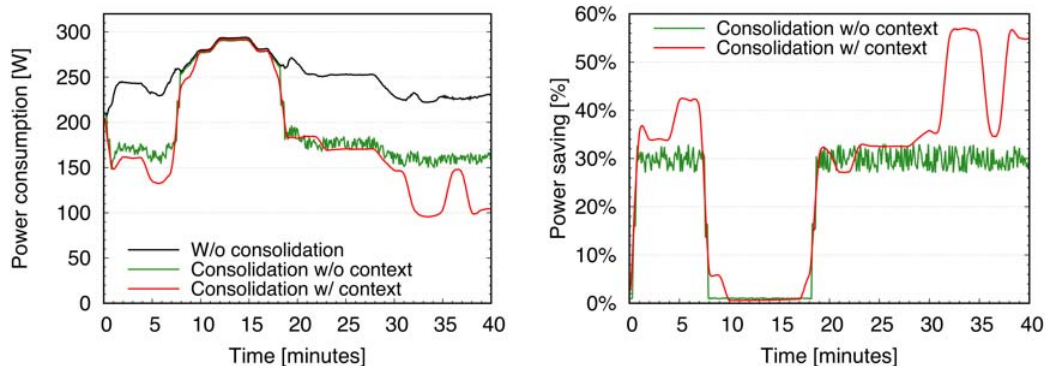


Figure 6. Comparison between consolidation without and with context information.

Without context information, VMs are treated uniformly by the consolidation procedure, independently of their actual utilization and role. This basically prevents servers from sleeping. In this case, only one redundant switch can be put to sleep. When context information is used, there is the opportunity for larger energy savings; this is only possible when there are idle VMs. Clearly, saving energy is only possible when the system is partially used; we note that, in both consolidation scenarios, power savings decreases with higher processing load. Overall, energy saved during the simulation amounts to about 21 percent and 28 percent on average without and with context information, respectively.

CONCLUSION

In this paper, we investigate the usage of more context in the consolidation process. Our approach leverages emerging software orchestration paradigms, which are able to automatically provide and update context information for elastic services. The specific contribution of this paper consists in the evaluation of the proposed concept in an experimental framework. We have demonstrated both the feasibility of the concept with small extensions to OpenStack, as well as the additional improvement in terms of energy savings.

Future work will address the missing gap in our framework, namely pricing models. The objective is a rewarding mechanism, which offers lower rates for idle VMs, so as to incentivize users to declare the proper context information for their virtual resources. Pricing models should find the best equilibrium between these factors, so as to maximize revenues for data center owners.

ACKNOWLEDGMENTS

This work was supported in part by the European Commission under the projects ARCADIA (contract no. 645372) and MATILDA (contract no. 761898).

REFERENCES

1. A. Carrega and M. Repetto, "Energy-Aware Consolidation Scheme for Data Center Cloud Applications," *29th International Teletraffic Congress (ITC 29)*, 2017, pp. 24–29.
2. A. Carrega et al., "OpenStack extensions for QoS and energy efficiency in edge computing," *Third International Conference on Fog and Mobile Edge Computing (FMEC)*, 2018, pp. 50–57.
3. L.A. Barroso and U. Hözlze, "The case for energy-proportional computing," *Computer*, vol. 40, no. 12, 2007, pp. 33–37.
4. S. Srikantaiah, A. Kansal, and F. Zhao, "Energy aware consolidation for cloud computing," *Proc. of the 2008 Conference on Power aware computing and systems (HotPower'08)*, 2008.
5. M. Mishra and A. Sahoo, "On theory of VM placement: Anomalies in existing methodologies and their mitigation using a novel vector based approach," *IEEE Int. Conference on Cloud Computing (CLOUD)*, 2011, pp. 275–282.
6. H. Shirayanagi, H. Yamada, and K. Kono, "Honeyguide, "A VM migration-aware network topology for saving energy consumption in data center networks," *IEEE Symposium on Computers and Communications (ISCC)*, 2012, pp. 460–467.
7. W. Fang et al., "VMPlanner: Optimizing virtual machine placement and traffic flow routing to reduce network power costs in cloud data centers," *Computer Networks*, vol. 57, no. 1, January 2013, pp. 179–196.
8. L. Wang et al., "Joint virtual machine assignment and traffic engineering for green data center networks," *ACM SIGMETRICS Performance Evaluation Review*, vol. 41, no. 3, 41, December 2013, pp. 107–112.

9. A. Stage and T. Setzer, "Network-Aware Migration Control and Scheduling of Differentiated Virtual Machine Workloads," *Proceedings of the ICSE Workshop on Software Engineering Challenges of Cloud Computing*, 2009, pp. 9–14.
10. B. Li et al., "Enacloud: An energy-saving application live placement approach for cloud computing environments," *IEEE International Conference on Cloud Computing (CLOUD '09)*, 2009, pp. 17–24.
11. B. Heller et al., "Elastictree: Saving energy in data center networks," *Proceedings of the 7th USENIX conference on Networked systems design and implementation (NSDI'10)*, vol. NSDI'10, 2010.
12. A. Beloglazov and R. Buyya, "OpenStack Neat: a framework for dynamic and energy-efficient consolidation of virtual machines in OpenStack clouds," *Concurrency and Computation: Practice and Experience*, vol. 27, no. 5, April 2015, pp. 1310–1333.
13. V. Cima et al., "Adding energy efficiency to Openstack," *Sustainable Internet and ICT for Sustainability (SustainIT)*, 2015, pp. 1–8.
14. M.E.M Diouri et al., "Your cluster is not power homogeneous: Take care when designing green schedulers!," *International Green Computing Conference (IGCC)*, 2013.

ABOUT THE AUTHORS

Alessandro Carrega is a research associate at CNIT. His main research topics are networking (energy-aware, performance optimization, and virtualization), software routers, and NFV and SDN (in particular, OpenFlow). He is an active reviewer for many international journals and conferences (IEEE and ACM). Contact him at alessandro.carrega@cnit.it.

Giancarlo Portomauro is a research fellow at the Electrical, Electronics, and Telecommunication Engineering and Naval Architecture Department (DITEN) at the University of Genoa. His main activities concern emulation of commutation of on-board satellite systems, reliable QoS network satellite for multimedia applications, and software for the satellite emulation system. Contact him at giancarlo.portomauro@edu.unige.it.

Matteo Repetto is a research associate at CNIT. His research interests include pervasive communications and mobility management, energy-efficient networking, SDN and network function virtualization, and cloud computing. Contact him at matteo.repetto@cnit.it.

Giorgio Robino is a software developer and research assistant at CNIT. His research interests include conversational AI, natural language understanding, and voice user interfaces. Contact him at giorgio.robino@cnit.it.